

ESP32-WROOM-32D Custom Development Board Hardware Design Documentation and Learning Notes V2.0

Version: V2.0

Author: Devansh Sharma (1st year Integrated MSc)

Institution: NISER, Bhubaneswar

This document covers the complete hardware design of the ESP32 custom development board using ESP32-WROOM-32D module version 2.0, including schematic design, PCB layout decisions, and component selection rationale.

TABLE OF CONTENTS

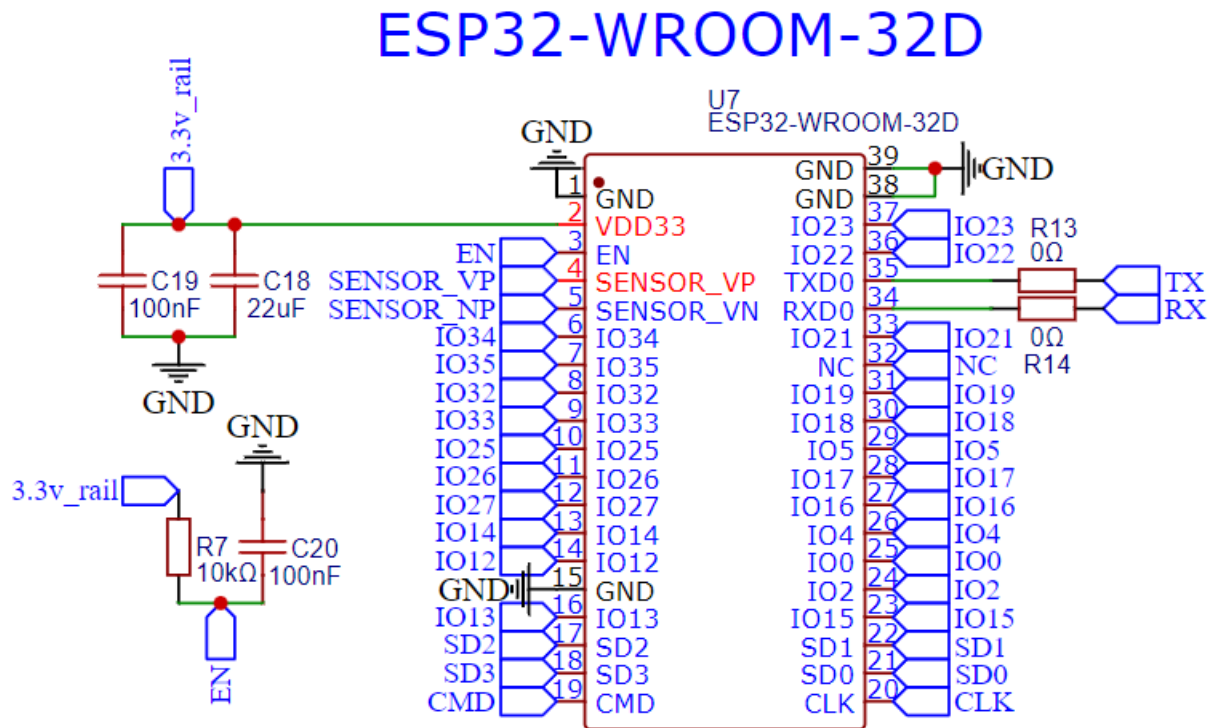
1) ESP32 WROOM 32D Module (Processor + Support Circuit)	4
1.1) The ESP32 WROOM 32D Module	4
1.2) Power Supply and Decoupling	5
1.3) Enable (EN) Pin and Reset Behaviour	5
1.4) Boot Strapping Configuration	6
1.5) Clock, Flash Power and RF (Inside the Module)	6
1.6) Grounding and Thermal Pad	6
1.7) Routing Considerations	7
1.8) Final Verdict	8
2) Flash Interface (Integrated in Module)	8
3) USB-UART Interface	9
3.1) USB to UART Conversion	9
3.2) USB Signal Change	10
3.3) Powering the USB-UART IC	10
3.4) Placement and Layout Consideration	10
3.5) Probing Points for Debugging	11
3.6) Other Control and Status Pins	11
3.7) Final Verdict	12
4) Auto Reset and Boot Control	12
4.1) Transistor Based Control	12
4.2) Working of the Circuit	13
4.3) Design and Layout Consideration	13
4.4) Test Points for Debugging	13
5) Power Block	14
5.1) Power Input Sources	15
5.2) Selection of 5V and Making of Final Rail	16
5.3) Production of Final 3.3V Rail	16

5.4) Trace Width, Power Distribution and Grounding	17
5.5) Test and Debug	17
5.6) Final Verdict	17
6) Module Place and Antenna Keep Out	17
6.1) Impact of Poor Placement	18
6.2) Final Verdict	18
7) Design Decisions	19
7.1) Buck Regulator over LDO for 3.3V Rail	19
7.2) CP2102N over CH340G for USB-UART	19
7.3) ESP32 WROOM 32D Module over Bare Chip	19
7.4) LM66100 Ideal Diode over Schottky for Power Selection	19
8) Components Brief Description	19
8.1) ESP32 WROOM 32D - WiFi + BLE Module	19
8.2) CP2102N-A02-GQFN28 - USB to UART Bridge	19
8.3) TPS62163DSGR - Buck Regulator (Vin to 5V)	20
8.4) TPS62162DSGT - Buck Regulator (5V to 3.3V)	20
8.5) LM66100DCKR - Ideal Diode Controller (x2)	20
8.6) USBLC6-2SC6 - USB ESD Protection	20
8.7) SS8050-G (x2) - NPN Transistors (Auto Reset Circuit)	20
8.8) Ferrite Bead (600Ω at 100MHz) - USB VBUS Filtering	20
8.9) Polyfuse - Overcurrent Protection	21
9) Brownout Triggering	21
10) XIP (Execute-In-Place)	21
11) USB to UART Conversion (How it Actually Works) and why	21
12) References	22
13) Version History	23
14) Results	23

1) ESP32 WROOM 32D Module (Processor + Support Circuit)

This section explains the ESP32 WROOM 32D module implementation, including the power pin, decoupling, enable and boot configuration, grounding, and the design constraint required for stable operation.

1.1) The ESP32 WROOM 32D Module:



The ESP32 WROOM 32D is a premade module from Espressif build around ESP32-D0WD chip. Along with the chip module already contains:

- 4MB SPI flash
- 40MHz crystal Oscillator
- RF matching network
- PCB trace antenna (MIFA)

Because the module has most things integrated, it only needs a clean 3.3V supply, proper EN and GPIO0 handling, a USB-UART interface for programing, and the correct placement of module antenna.

1.2) Power supply and Decoupling:

In this design, the module is powered from the 3.3v_rail. As per the Espressif datasheet, the module needs 3.0V to 3.6V, with a power supply that can give atleast 500mA. The 3V3 pin is decoupled using:

- 100nF capacitor: high-frequency noise reduction
- 22uF capacitor: transient current support during WIFI bursts

Placement rules followed:

- Capacitors are placed as close as possible to the 3V3 pin
- Direct, short connection to ground plane using vias
- Minimal loop area

This way of layout practice is necessary for ESP32 as it exhibits dynamic current behaviour as follows:

- Low current during idle
- Sudden high current during WIFI transmission

Without proper decoupling:

- Voltage dips occur
- Brownout detector triggers, along with random resets and instability

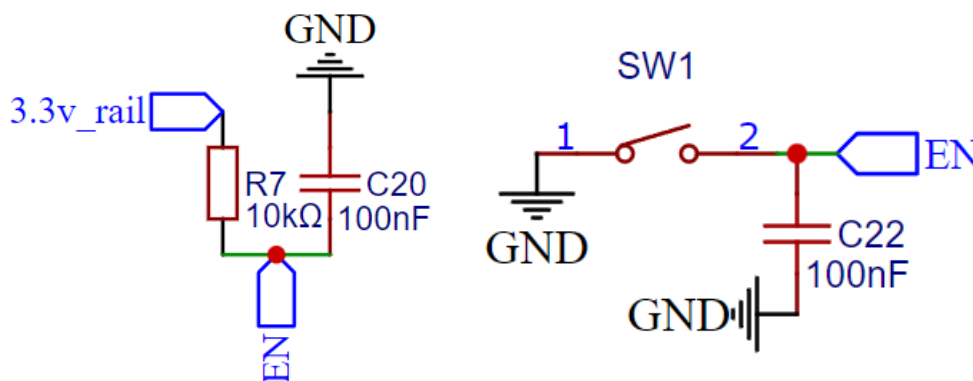
1.3) Enable (EN) Pin and Reset Behaviour:

The EN pin of the module acts as the global enable/reset signal for the ESP32 inside it.

- EN must be HIGH for normal operation
- Pulled up to the 3.3v_rail using a 10kΩ resistor
- Connected to the auto reset circuit (later in the document) and to the RESET push button

A 100nF capacitor is added between EN and GND to filter noise and give a power-up delay. One more 100nF capacitor is placed across the RESET push button for debouncing.

If EN is not handled properly, then the chip may reset randomly, also startup behaviour becomes unreliable.



1.4) Boot Strapping Configuration:

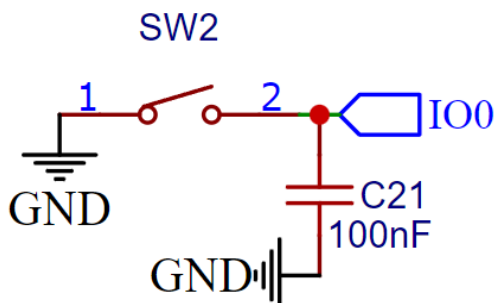
ESP32 boot mode is determined by strapping pins, by default GPIO0. The following is the boot behaviour:

- GPIO0 LOW: enters flashing mode
- GPIO0 HIGH: normal execution

What it does is, that it keeps GPIO0 pulled up by default and also controlling GPIO0 via DTR during programming.

This ensures the automatic entry into bootloader, and normal operation after reset.

If the strapping pins are kept floating or incorrectly biased, then, boot failures occur, sometimes wrong mode gets selected, programming becomes unreliable. In short making entire reset system unpredictable and unreliable.



1.5) Clock, Flash Power and RF (Inside the Module):

Unlike the next bare chip design, in the ESP32 WROOM 32D, the 40MHz crystal, the flash along with its VDD_SDIO supply, and the RF matching all are inside the module and need not to be worried about.

1.6) Grounding and Thermal Pad:

The module has three GND pins (pin 1, 15 and 38) and an exposed ground pad under the module. All of them are connected to the ground plane with multiple vias. The ground pad serves as:

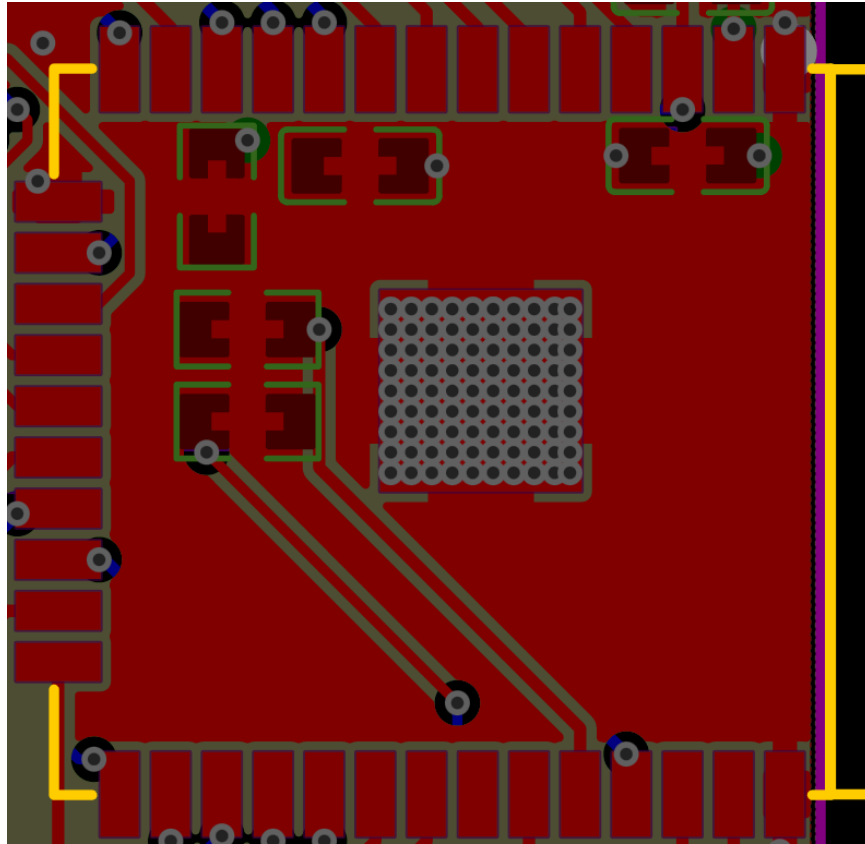
- Thermal dissipation
- Central ground reference
- Noise path return

A solid GND plane is maintained under the module. To avoid GND plane (or any plane) below antenna, it is placed out of board area.

This continuous solid grounding is very important as without it there is:

- Increment in impedance in return path

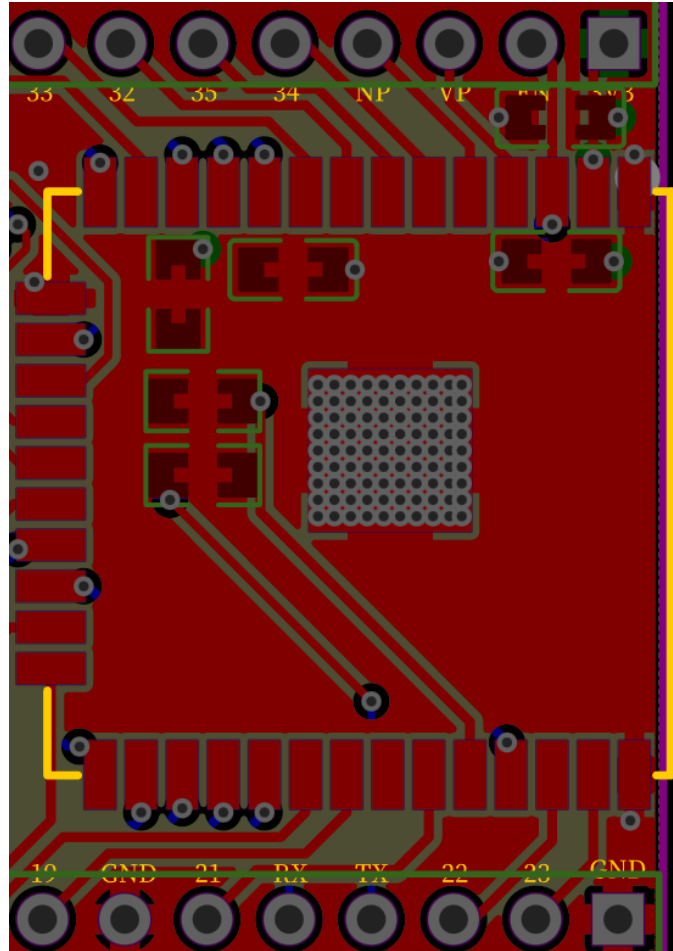
- Higher EMI
- Unstable RF behaviour



1.7) Routing Considerations:

While routing signals and placement of board the following are taken care of:

- Avoided placement of antenna over copper area because i placed it over the board edge
- Kept the 3.3V supply and decoupling paths short by directly using vias
- Separated noisy signals like buck conversion from module



1.8) Final Verdict:

Despite flash, clock, RF are integrated in module itself, still the module core is dominated by interaction between power integrity, reset behaviour, boot configuration, and grounding. These factors are tightly coupled, as a mistake in one area can cause failure in another.

For example:

- Poor decoupling results in voltage dip, which results in reset resulting in boot failure
- Too small EN delay can make the ESP32 start before the supply is stable
- Incorrect GPIO0 bias can result in the system stuck in the wrong mode

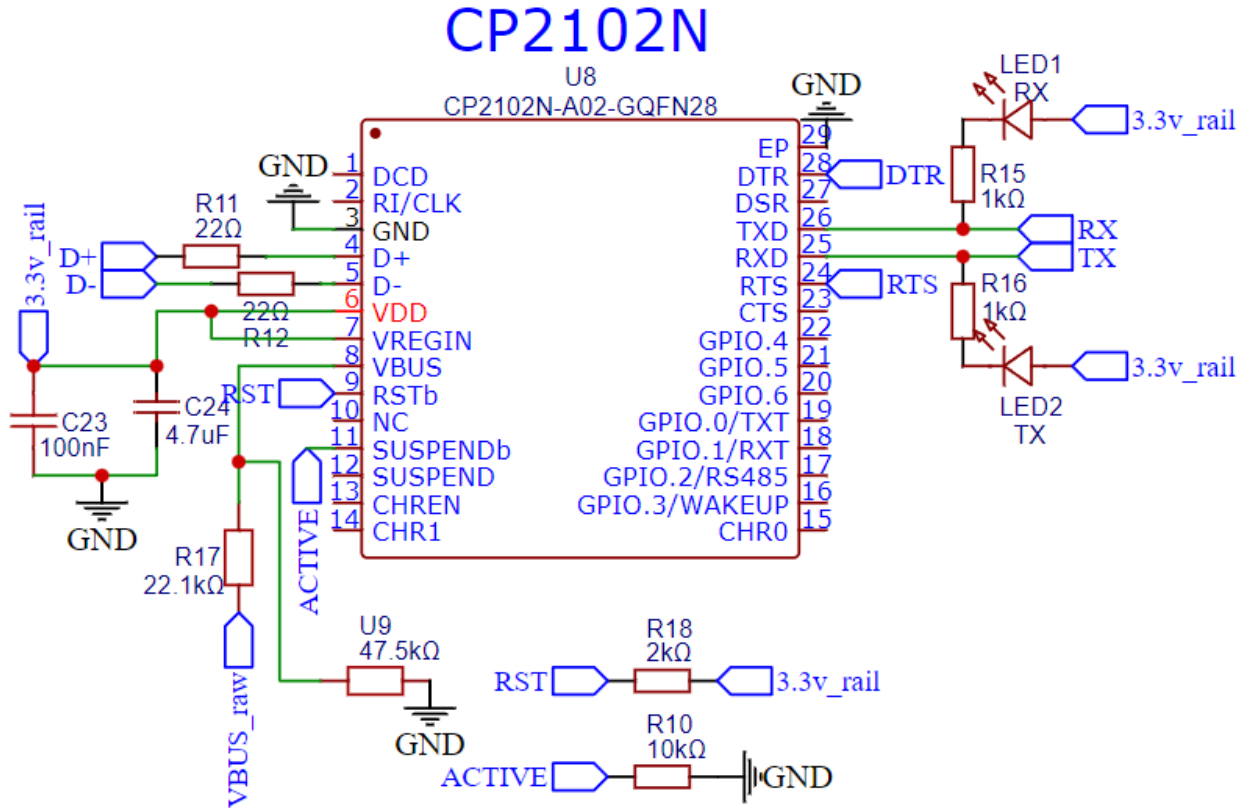
So, the ESP32 module should be treated as a system and not just a component

2) Flash Interface (Integrated in Module)

The ESP32 WROOM 32D has the SPI flash integrated inside the module (4MB in this module), which is connected to the ESP32 through the SD_DATA, SD_CLK and SD_CMD pins (GPIO6 to 11). Because of this no external flash is required.

These six pins (SD0, SD1, SD2, SD3, CMD and CLK) are still broken out on headers, but as per Espressif datasheet they are connected to the internal flash and are not recommended for any other use. These pins are kept on the headers only for probing and should not be used as GPIO.

3) USB-UART Interface



The USB-UART interface is used to establish communication between the computer and ESP32. In this design, the 28 pin CP2102N USB to UART Bridge is used, which converts USB signals into UART signals required by ESP32. This section is not only responsible for serial communication but also enables automatic reset and boot control during programming. So, it plays a critical role in both functionality and usability of the board.

3.1) USB to UART conversion:

The CP2102N acts a bridge between USB (D+/D-) and ESP32 (TX/RX). In this design, the D+/D- from USB is first taken to an ESD protection diode (USBL62SC6) and to differential resistors of 22ohms to match impedance (more details in components section).

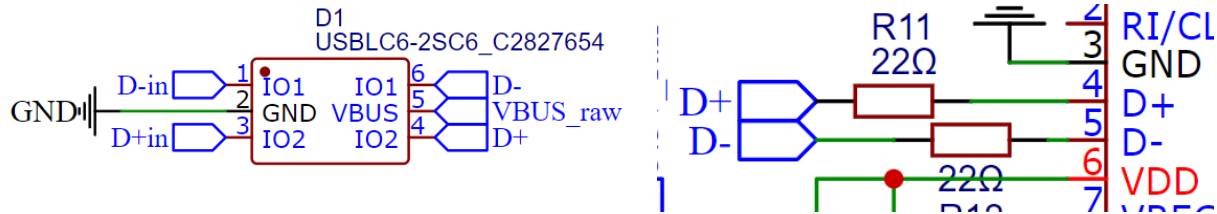
Its connections are made as follows:

- TX (CP2102N) to RX (ESP32)
- RX (CP2102N) to TX (ESP32)

This allows serial communication, code upload, and debug logging. If this connection is incorrect, then there is no communication, upload fails, and serial monitor does not work.

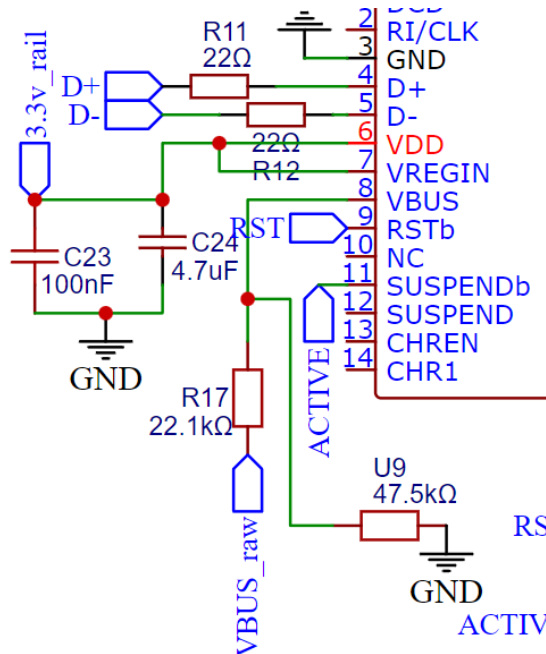
3.2) USB Signal Change:

The signal path is from USB connector to ESD protection to Series Resistors to CP2102N. This order is important because ESD diode protects from voltage spikes, and series resistors help in signal matching with matching the impedance to ~90 ohms. If order is changed/alterd ESD can reach the IC, signal quality degrades, and USB enumeration may fail.



3.3) Powering the USB-UART IC:

In this design, CP2102N is being powered at 3.3V, or we can say with common 3.3V plane for entire board (Check the power section for detailed path of power). Proper decoupling is present at VCC pins of IC (in this design 100nF with 4.7uF). Without this decoupling USB connection is unstable with random disconnects.



3.4) Placement and Layout Consideration:

- CP2102N is placed close to USB connector after ESD diode
- Short D+/D- routing

- UART lines routed cleanly to ESP32

This altogether reduces signal loss and noise increment.

3.5) Probing Points for Debugging:

To improve debugging and bring-up reliability, test points are provided on key USB-UART and control lines: On DTR, RTS. These allows:

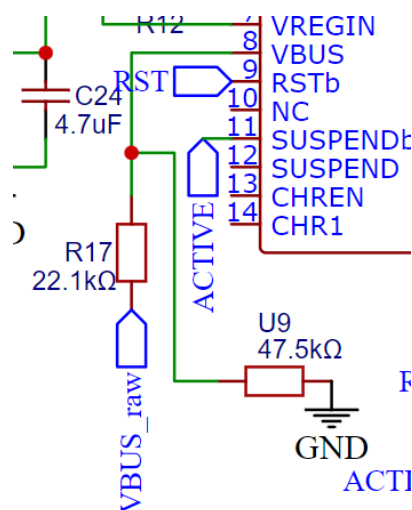
- Verifying UART communication between ESP32 and USB interface
- Observing reset and boot signals (DTR/RTS behaviour)
- Debugging upload failures and communication issues

DTR and RTS are not brought out to any header, so they can only be probed at the base resistors of the auto reset transistors.

3.6) Other Control and Status Pins:

In addition to UART and reset control signals, a few auxiliary pins of the CP2102N USB to UART Bridge are used in this design to ensure proper operation and status indication:

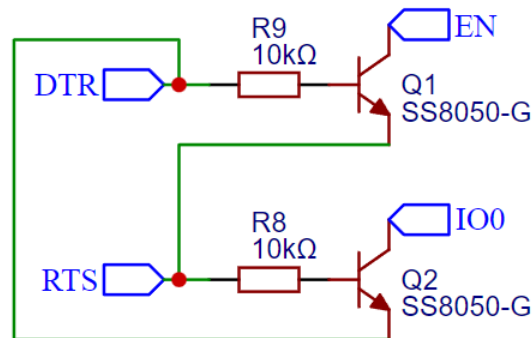
- VBUS: connected to the USB 5V line (VBUS_raw) through a resistor divider network of 22.1k Ω and 47.5k Ω . This allows the IC to detect USB connection presence and ensure proper USB enumeration.
- VREGIN & VDD: the internal regulator pin (VREGIN) is powered from the 3.3V rail, with proper decoupling capacitors (100nF and 4.7uF). This ensures stable operation of the internal circuitry.
- RSTb: pulled HIGH using a resistor of 2k Ω to keep device in normal operating mode. Pulling this LOW would reset the USB-UART IC.
- SUSPENDb: This pin indicate the USB suspend state. Although as per official reference it is connect to 10k Ω resistor, but is not used anywhere in the design.



3.8) Final Verdict:

The USB-UART section is not just for communication but also controls the programming flow of ESP32. Correct routing, proper reset wiring, a signal integrity are essential for reliable operation. Even if ESP32 core and flash are perfectly designed, a mistake in this section can make the board unusable for development.

4) Auto Reset and Boot Control



The auto reset circuit is responsible for automatically placing the ESP32 into programming mode during code upload. This removes the need for manual interaction (pressing BOOT and RESET buttons) and enables seamless flashing through USB.

This circuit uses control signals from the CP2102N USB to UART Bridge, specifically DTR and RTS, along with transistor-based switching to control the ESP32's EN (reset) and GPIO0 (boot) pins. Because ESP32 enters programming mode only when GPIO0 is LOW and EN (reset) is toggled. This sequence must happen in a controlled manner during flashing. The auto reset circuit ensures this happens automatically whenever firmware upload is initiated.

In this circuitry, DTR (Data Terminal Ready) which controls GPIO0 and RTS (Request to Send) which controls EN, are the signals used. These signals are driven by USB-UART IC during the upload.

4.1) Transistor Based Control:

Two NPN transistors (SS8050) are used in the design to control the Esp32 pins:

- One transistor for EN (reset)
- One transistor for GPIO0 (boot)

Why transistors are used:

- CP2102N outputs are logic level signals
- The ESP32 requires controlled pull-down behaviour

- Transistors provide, signal inversion, proper switching and isolation between circuits

4.2) Working of the Circuit:

During normal operation, EN is HIGH meaning that ESP32 runs normally and GPIO0 is HIGH meaning normal boot mode.

During programming, DTR pulls GPIO0 LOW, along with RTS toggling EN (reset pulse) which makes ESP32 enter into bootloader mode. After upload signals return HIGH and ESP32 runs uploaded program.

4.3) Design and Layout consideration:

- A 10k pull up resistor is used on EN to ensure a stable default HIGH state, along with a 100nF capacitor on EN to reduce incoming noise. GPIO0 has no external pull up and relies on the internal pull up of ESP32
- Base resistors (10k) are used on DTR and RTS lines while connecting them to transistors to limit current into bases
- Proper routing ensuring minimal noise coupling into reset lines

If these design considerations are not implemented:

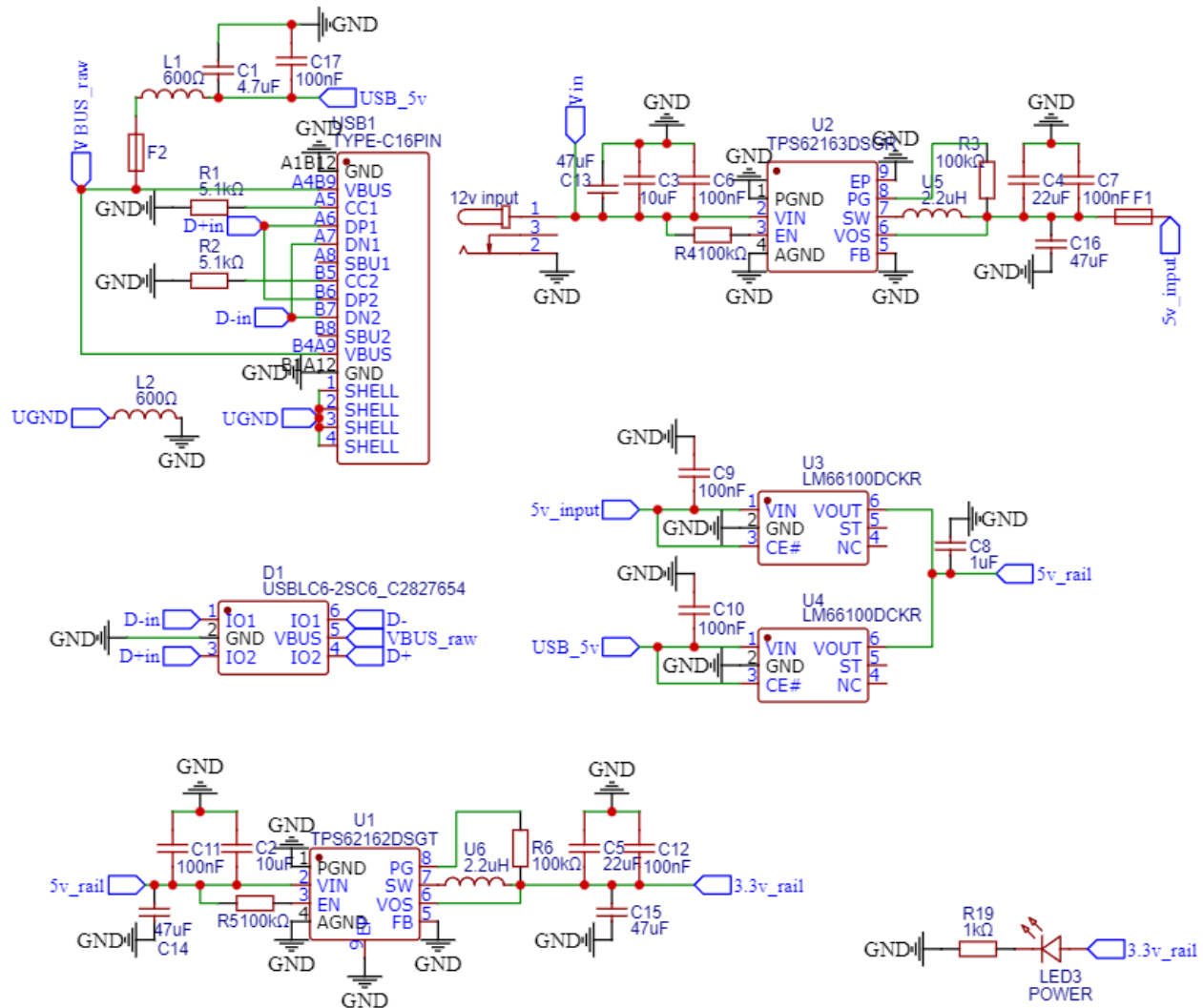
- ESP32 does not enter flashing mode
- Upload fails repeatedly
- Manual reset may still work (indicating circuit issue)

4.4) Test points for Debugging:

No dedicated points are provided for this circuit on the board. EN and GPIO0 can be probed from the headers, while DTR and RTS can only be probed at the base resistors of transistors. This makes debugging of auto reset behaviour harder, as the signal transitions during upload cannot be checked easily.

5) Power Block

POWER_BLOCK

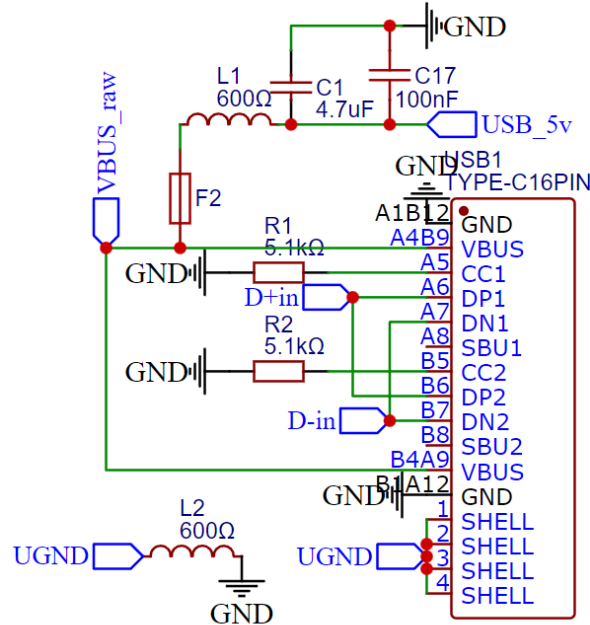


The power block is designed to accept input from multiple sources and generate stable voltage rails required for stable system operation. This design supports both USB-based power and external VIN supply, allowing flexibility with ensuring proper protection, regulation, and power integrity across the board. The system ultimately derives a 3.3V rail, which powers the ESP32, external flash memory, USB-UART interface, and associated logic circuitry. To achieve this, a multi-stage power architecture is used, consisting of input protection, power selection, intermediate regulation, and final 3.3V conversion.

5.1) Power Input Sources:

Power can be supplied through either:

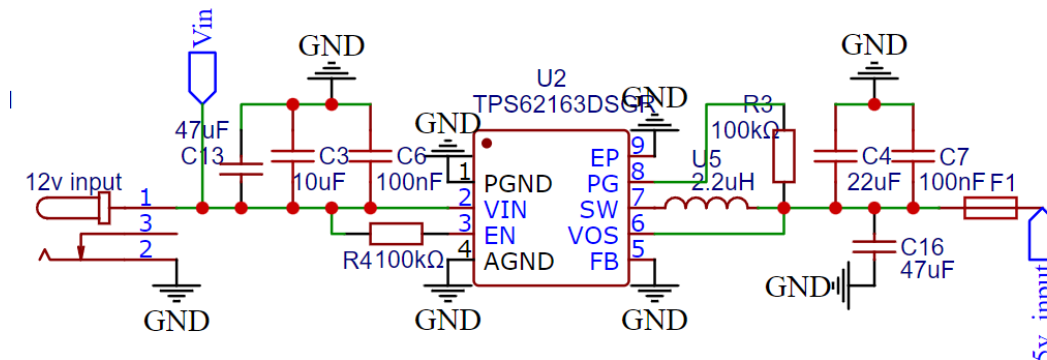
- **USB Type-C (5V_VBUS):**



The USB path includes protections like a polyfuse followed by 600Ω at 100MHz ferrite bead on common VBUS line from USB-C, which protect against overcurrent conditions and surpasses high frequency noise entering from the cable to the USB-C. decoupling capacitors of 100nf and 4.7uF are used after the protection to produce 5V rail from the USB (named as USB_5v).

Also 5.1kΩ resistors are used on both CC pins to allow proper detection of USB device in PC and supplying 5V to it.

- **External VIN input (nominally higher voltage, example 12V):**

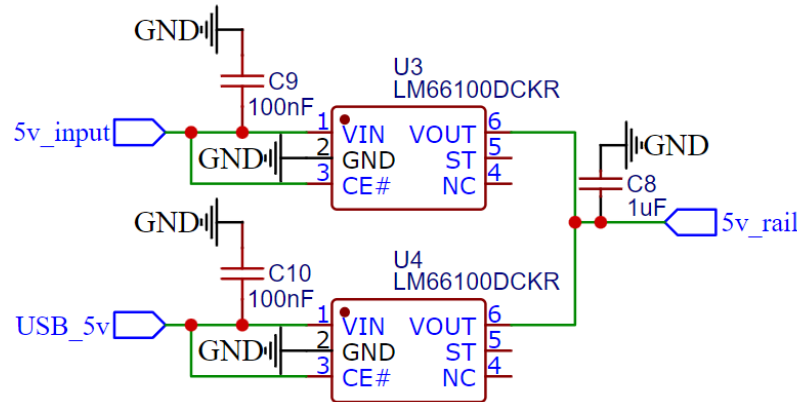


The external input comes from a DC barrel jack, and also from the Vin pin of 2 pin header. The input is decoupled using 47uF, 10uF, and 100nF capacitors and is given directly to the buck

regulator (TPS62163), whose EN is pulled up to the input using a 100k Ω resistor. Along with a 2.2uH inductor, output decoupling of 22uF and 100nF, a 100k Ω pull up on the PG pin, and a polyfuse, giving out 5V rail (5v_input).

Here, in both input and output a 47uF bulk decoupler is also added in order to counter any current spikes during ESP32 WIFI outburst.

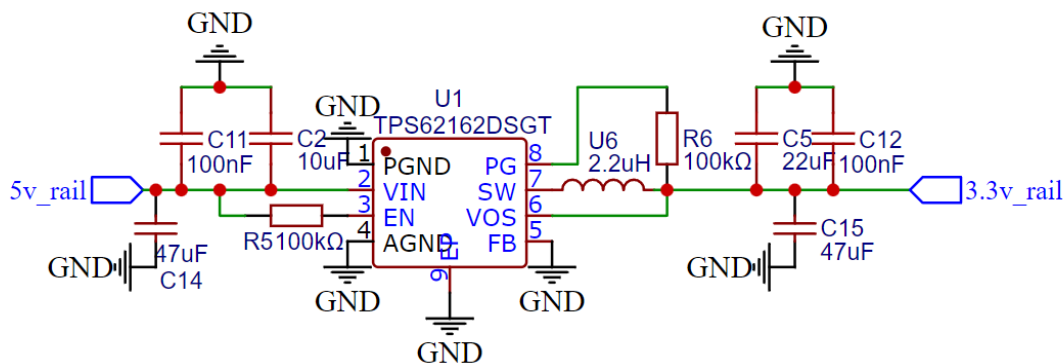
5.2) Selection of 5V and making of final rail:



Now as two different 5V rails are produced, one as USB_5v and other as 5v_input. These both 5V rails are then connected to ideal diode-ORing using two LM66100DCKR, which automatically selects the higher 5V and prevent any back-feeding of one rail into other. Thus, in this way a final 5V rail (named 5v_rail) is produced.

Also, appropriate decoupling is also done at each input and output point.

5.3) Production of final 3.3V rail:



The final 5v_rail is then passed through another buck (TPS62162), with proper decoupling at both input and output, along with a 2.2uH inductor, and on both sides a 100k Ω pull up resistor. Here no protection is added, as the 5V going in the input is well protected from start.

Also, those 47uF decoupler are also added here to counter any current spikes of ESP32 during WIFI bursts.

Although, there was a choice between a 3.3V LDO and a 3.3V buck. Despite the fact that due to fast switching of buck, extra noise may get added. I chose buck over LDO as buck are better and efficient to handle current spikes along with, using the fact that for an ESP32 development board, this minor noise addition will not be much harmful.

Decoupling and filtering is very important for the ESP32, as insufficient decoupling can result in voltage dips, leading to resets or unstable behaviour during high activity period.

5.4) Trace Width, Power Distribution and Grounding:

In all steps of power rail generation, traces are ensured to be sufficiently wide, and also it is taken care that traces are short enough to minimize resistive loss.

The regulated 3.3V is then made into a power plane (layer 3), and power to all components like ESP32, CP2102N, etc, is given by either via to power plane at the pad edge or by a short trace and then via to power plane, it is done to make sure to ensure shortest path to allow minimal resistive loss and prevent unnecessary noise addition.

Also, a continuous GND plane is used (layer 2), which is connected to components in a same way that for power plane. This ensures the shortest possible return path for components, which in turn provides a low-impedance path, which improves overall stability and reduce noise coupling.

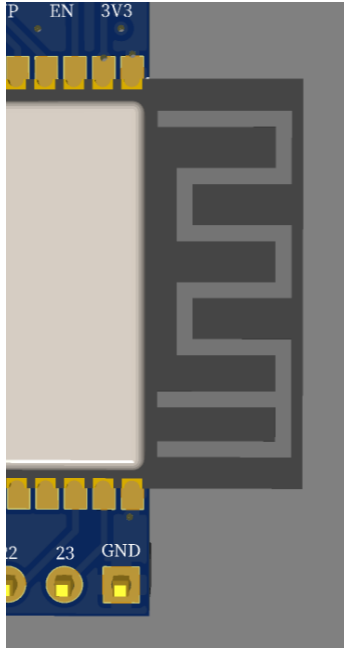
5.5) Test and Debug:

For test and debug support, no dedicated test points are provided on the power rails in this board. The 3.3v_rail and GND can be measured from the headers, and Vin from the 2 pin header, while the 5v_rail can only be measured at the capacitor pads. This makes bring up slightly harder, but still allows verification of regulator performance and diagnosis of power related issues.

5.6) Final Verdict:

The power architecture combines efficiency, protection, and stability through a dual-stage buck conversion approach. By using switching regulators for both intermediate and final stages, the design avoids excessive power loss while maintaining reliable operation under varying load conditions. Proper filtering, grounding and layout considerations ensures that the generated rails meet the requirements of the ESP32 and associated circuitry.

6) Module Place and Antenna keep out



The ESP32 WROOM 32D has an onboard PCB antenna (MIFA) along with the RF matching network, both already tuned by Espressif. But still as per Espressif guidelines, the module is placed at the edge of the board, with the antenna portion of the module extended out of the board edge.

As antenna is over the board edge, there is naturally no copper below it, no components near the antenna.

6.1) Impact of poor placement:

If the module is placed in the middle of the copper board, or copper is poured below its antenna, the system may still perform but with degraded performance like:

- Reduced wifi/bluetooth range
- Unstable connections
- Increased packet loss
- Higher power consumption due to retransmissions

These issues are often difficult to debug because the system appears to be operational but performs poorly.

6.2) Final Verdict:

With a module, the antenna design problem becomes an antenna placement problem. As long as the module sits at the board edge with a proper keep out region, the pre tuned module antenna gives wireless performance without any RF tuning on the board.

7) Design Decisions

7.1) Buck regulator (TPS62163) over LDO for 3.3V rail:

A switching buck regulator was chosen over an LDO, despite the fact buck will add some noise due to high frequency switching because of the two primary reasons. First, efficiency: unlike an LDO which dissipates excess voltage as heat, a buck regulator maintains high efficiency across varying load conditions. Second, the ESP32 exhibits sudden high current demands during WIFI transmission bursts, and a buck regulator handles these transient spikes significantly better than an LDO, reducing the risk of voltage dropping and brownout triggering.

7.2) CP2102N over CH340G for USB-UART:

The CP2102N was selected as the USB-UART bridge following the official ESP32 reference design. It provides reliable auto-reset and boot control support through DTR and RTS signals, which is crucial for seamless programming without manual button interaction.

7.3) ESP32 WROOM 32D module over bare chip:

The ESP32 WROOM 32D module was chosen over a bare ESP32 chip, as it already contains the flash, 40MHz crystal, RF matching network and PCB antenna, all tested and pre certified by espressif. This removes the hardest part of an ESP32 design (flash routing, crystal layout, RF design) and focus stay on the power, USB-UART and auto reset circuit. The trade off is less flexibility, as the flash size and antenna are fixed by the module.

7.4) LM66100 Ideal Dide over Schottky for Power Selection:

The LM66100 ideal diode controller was chosen over a conventional Schottky diode or combining USB and external Vin rails. Having used it previously, its near-zero forward voltage drop makes it significantly more efficient than a Schottky, and it inherently prevents back-feeding between the two power sources, ensuring clean power selection without energy loss.

8) Components Brief Description

8.1) ESP32 WROOM 32D (Espressif System) - wifi + BLE module:

A pre certified module built around the ESP32 D0WD, a dual core 32-bit LX6 microcontrollers with integrated WIFI (802.11b/g/n) and Bluetooth (classic + BLE). The module integrates 4MB SPI flash, a 40MHz crystal, and RF matching network and a PCB antenna, and operates from a single 3.0V to 3.6V supply.

8.2) CP2102N-A02-GQFN28 (Silicon Labs) - USB to UART Bridge:

A 28 pin USB to UART bridge IC that converts USB signals to UART for serial communication with ESP32. Chosen for its reliable auto-reset and boot control support through DTR and RTS

signals, enabling seamless automatic programming without manual button interaction. Provides stable driver support across all major operating systems.

8.3) TPS62163DSGR (Texas Instruments) - Buck Regulator (Vin to 5V):

A high-efficient synchronous step-down buck regulator used to convert the external Vin supply (nominally 12V) down to 5V. Chosen for its high efficiency, compact package, and ability to handle the input voltage range expected from external supply. Operates with a 2.2uH inductor, 100k Ω pull up resistors on EN and PG and appropriate decoupling to produce a clean 5V rail (5v_input) from the external path.

8.4) TPS62163DSGT (Texas Instruments) - Buck Regulator (5V to 3.3V):

A high-efficiency synchronous step-down buck regulator used to convert the final 5V rail down to 3.3V rail for powering the entire board. A buck was chosen over an LDO for two reasons, higher efficiency across varying loads, and superior handling of sudden high current demands during ESP32 WIFI transmission bursts, reducing the risk of voltage drop and brownout triggering. Operating with a 2.2uH inductor, 100k Ω pull up resistors on EN and PG and 47uF bulk decoupling to counter current spikes.

8.5) LM66100DCKR (Texas Instruments) - Ideal Diode Controller (x2):

An ideal diode controller with near zero forward voltage drop, used in pairs for power rail ORing between USB_5v and 5v_input rails. Automatically selects the higher available 5V source while preventing back-feeding from one rail into the other. Chosen over a Schottky diode due to its significant lower voltage drop and active back-feed prevention, resulting in more efficient and reliable power selection.

8.6) USBLC6-2SC6 (STMicroelectronics) - USB ESD Protection:

A low capacitance ESD protection diode array specially designed for USB signal lines. Placed at the USB input before the CP2102N to protect the IC from electrostatic discharge events on the USB connector. Its low capacitance ensures minimal impact on USB signal integrity, which is critical for reliable USB enumeration and communication.

8.7) SS8050-G (x2) - NPN Transistors (Auto Reset Circuit):

NPN transistors used in the auto reset and boot control circuit to switch the ESP32's EN and GPIO0 pins based on DTR and RTS signals from CP2102N. Transistors are used instead of direct connection because the CP2102N outputs are logic level signals, while the ESP32 pins require controlled pull-down behaviour with proper isolation. Provides signal inversion and clean switching needed for reliable automatic programming.

8.8) Ferrite Bead (600 Ω at 100MHz) - USB VBUS Filtering:

A ferrite bead placed on the VBUS line from the USB connector. Suppresses high frequency noise entering from the USB cable into the board power path, acting as a low-pass filter for the

power line. The 600Ω impedance at 100MHz ensures effective attenuation of switching and cable-induced noise while passing DC without significant loss.

8.9) Polyfuse - Overcurrent Protection:

Two resettable polyfuses are used for overcurrent protection, one on the USB VBUS path and one after the Vin to 5V buck regulator.

Automatically resets after the fault is removed, unlike a one-time fuse.

9) Brownout Triggering

ESP32 has an internal voltage monitor. If supply voltage drops below a threshold (around 2.5-2.7V), even momentarily, ESP32 automatically resets itself. This is called brownout reset. It's a protection mechanism as running at insufficient voltage causes unpredictable behaviour, so it's safer to just reset.

During WIFI transmission, current spikes can cause momentary voltage dip on the rail. If decoupling is insufficient, that dip crosses the brownout threshold which results in automatic reset overall resulting in random crashing.

That's why decoupling and buck over LDO matters directly for this.

10) XIP (Execute-In-Place)

Normally in a microcontroller, firmware is copied from flash into RAM first and then executed from RAM. This requires RAM to be large enough to hold the entire program.

XIP is different. The processor reads and executes instructions directly from flash memory, without copying them to RAM first. This means:

- RAM is freed up for actual data and variables
- Larger programs can run than RAM alone would allow
- Flash becomes part of the execution path, not just storage

This is why the flash interface is boot-critical in ESP32. If flash communication is unstable even for a moment, instruction fetch fails, and the processor crashes. It's not just reading stored data; it's actively running code from there in real time.

In this board, the flash is inside the module, so this interface is already designed by Espressif. But XIP is also the reason why the SD0 to SD3, CMD and CLK pins on headers must never be used as GPIO, as disturbing them even for a moment stops the ESP32 from fetching its instructions.

11) USB to UART Conversion (How it actually works) and why CP2102N is needed

The CP2102N acts as a protocol bridge between two fundamentally different communication standards, USB and UART.

USB does not transmit data as simple logic signals. Instead, it uses differential signalling on two lines, D+ and D-. Data is encoded as the difference between these two lines, when D+ is high and D- is low it represents one logic state, and when D- is high and D+ is low it represents the other. This differential approach makes USB inherently noise resistant and capable of operating at high speeds. Additionally, USB is a packet-based protocol, meaning data is wrapped in structured packets with handshaking, error checking, and device enumeration, making it significantly more complex than simple serial communication.

The ESP32 on the other hand communicates via UART, a simple two wire protocol (TX/RX) where data is transmitted as plain voltage levels at a fixed baud rate. High voltage is logic 1, low voltage is logic 0. No differential signalling, no packets, no handshaking.

These two protocols are fundamentally incompatible and cannot communicate directly

The CP2102N internally contains two separate engines:

USB Engine: Receives the D+/D- differential signals, decodes the USB protocol, handles enumeration with the host computer, extracts the raw data bytes from USB packets, and manages all USB complexity internally. The computer simply sees it as a standard virtual COM port.

UART Engine: Takes the extracted data bytes from the USB engine and transmits them as simple UART signals on the TX pin to the ESP32 and vice versa receives UART data from ESP32 on RX pin and packages it back into USB format for the computer.

These two engines are internally connected inside the CP2102N, making the entire protocol translation invisible to both sides. The computer sees a USB device, the ESP32 sees plain UART. Neither needs to know about the other protocol.

This is why a USB-UART bridge IC is necessary, without it, the computer has no way to communicate with the ESP32, making firmware upload, serial monitoring, and debug logging impossible.

In this design, the D+/D- lines from the USB Type-C connector are first passed through the USBLC6-2SC6 ESD protection diode, then through 22Ω series resistors for impedance matching to approximately 90Ω differential impedance, and finally into the CP2102N. This order is critical, ESD protection must come before the IC to protect it from voltage spikes, and series resistors ensure signal integrity for reliable USB enumeration and communication

12) Reference

1) ESP32 Official Schematic diagram:

https://dl.espressif.com/dl/schematics/esp32_devkitc_v4-sch.pdf

2) ESP32-WROOM-32 Datasheet:

https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf

3) TPS62162 and TPS62163 Datasheet:

<https://www.ti.com/lit/ds/symlink/tps62160.pdf>

4) LM66100 Datasheet:

<https://www.ti.com/lit/ds/symlink/lm66100.pdf?ts=1776595592249>

5) CP2102N Datasheet:

<https://www.silabs.com/documents/public/data-sheets/cp2102n-datasheet.pdf>

6) USBLC6-2SC6 Datasheet:

<https://www.st.com/resource/en/datasheet/usblc6-2.pdf>

7) For components like Crystals and anything else, Datasheet from its LCSC is used while placing components in EasyEDA

13) Version History

Version 2.0 was final PCB layout as version 1.0 was not seem to be enough clean and good layout.

14) Results

- Achieved a **functional first hardware implementation**, although there is a unresolved slow power start up issue resulting in manual boot when connected to supply for one time
- Board is working normally as expected under DIO and 40 MHz flash operation
- Detects Wi-Fi ranging from -9dBm to -95dBm